Техническая документация: приложение Gelang Asistif

Версия 0.1, 16 октября 2026. Репозиторий: https://github.com/IvanFrost23/gelang-asistif-app. Документ описывает текущее состояние после шага 1 плана и целевую архитектуру для последующих шагов; будущие части помечены словом «план».

1. Назначение и границы

Приложение-компаньон к ассистивному браслету для глухих детей 7–12 лет. Управляется родителем или учителем. Задачи: профили детей, настройки категорий звуков, распознавание звуков с микрофона, отправка кода на браслет, история событий, guidebook и режим тренировки.

Железо в проекте не собирается. Bluetooth-подключение симулируется. Роль браслета на демонстрации играет сам телефон: анимированный браслет на экране и Vibration API на Android.

2. Стек

Слой	Решение	Причина
UI	React 18, TypeScript	знакомый стек, переиспользование наработок
Сборка	Vite 6	закреплено из-за Node 22.2 на машине разработки
Роутинг	react-router-dom 6	
Состояние (план)	zustand + localStorage	работает офлайн, переживает перезагрузку
Распознавание речи (план)	Web Speech API, язык id-ID	ловит имя ребёнка; доступен в Chrome без библиотек
Классификация звуков (план)	YAMNet через TensorFlow.js	готовая модель, 521 класс, работает локально
Вибрация	Navigator.vibrate	Android Chrome
Аккаунты и хранение (план)	Firebase Auth (email), Firestore	вход по email по требованию книги проекта
Хостинг (план)	Firebase Hosting	HTTPS, нужен для микрофона
Нативная обёртка (опция)	Capacitor	если потребуется APK

3. Структура проекта

```
projectT/
  docs/                книга проекта, карточка, руководства, скриншоты
  src/
    app/routes.ts      все пути приложения в одном объекте
    components/        Screen (шапка + контент + навигация), BottomNav, MenuRow,
Icons
    screens/           Home, Settings, History, Help, Stub (заглушки)
    App.tsx            таблица маршрутов
    index.css          тема: переменные цветов, карточки, кнопки, навигация
  index.html           lang="id", viewport под телефон
```

План по мере шагов: `src/domain/signals.ts` (категории, цвета, паттерны), `src/components/Bracelet.tsx`, `src/hooks/useVibrate.ts`, `src/store/` (zustand), `src/listen/` (микрофон, речь, YAMNet), `src/simulator/`.

4. Маршруты и навигация

Путь	Экран	Статус
`/`	Beranda: профиль, статус браслета, ручной вызов, меню	каркас
`/pengaturan`	Pengaturan: категории звуков, чувствительность	каркас
`/riwayat`	Riwayat: история событий	каркас
`/bantuan`	Bantuan: ссылки на guidebook и тренировку, о проекте	каркас
`/masuk`	вход по email	заглушка, шаг 3 и 9
`/profil`	профиль ребёнка, имена	заглушка, шаг 3
`/sambungkan`	подключение браслета	заглушка, шаг 3
`/latihan`	режим тренировки	заглушка, шаг 3
`/panduan`	guidebook	заглушка, шаг 6

Нижняя навигация фиксирована внизу, четыре вкладки как на карточке прототипа. Макет ограничен шириной 430px и центрируется на широком экране.

5. Домен сигналов (план, шаг 2)

Единственный источник правды для кодов: один модуль с четырьмя записями.

```
 type Category = 'name' | 'danger' | 'bell' | 'home'
interface Signal {
  category: Category
  label: string          // индонезийское название
  color: string          // CSS-цвет LED
  blink: boolean         // мигание для danger
  pattern: number[]      // мс: [вибрация, пауза, вибрация, ...] для
navigator.vibrate
  alwaysOn: boolean      // danger нельзя отключить
}
```

Паттерны по книге: name `[150, 100, 150]`, danger `[600, 200, 600, 200, 600]` с повтором, bell `[400]`, home `[120, 80, 120, 80, 120]`. Компонент браслета и Vibration API читают один и тот же массив, поэтому анимация и реальная вибрация всегда совпадают.

6. Конвейер событий (план, шаги 5, 7, 8)

Все источники порождают одинаковое событие `{ category, source, confidence, at }`, где source ∈ `simulator | manual | training | speech | yamnet`.

1. Источник создаёт событие.
2. Фильтр настроек: категория включена (danger всегда), confidence не ниже порога из чувствительности.
3. Антидребезг: события одной категории не чаще раза в 3 секунды.
4. Событие записывается в историю и отправляется «браслету»: анимация плюс вибрация.

Симулятор нужен как гарантированный путь демонстрации без микрофона и сети.

7. Режим Dengar (план, шаги 7 и 8)

Один `MediaStream` с микрофона делится между двумя распознавателями.

Имя. `SpeechRecognition` с `lang = 'id-ID'`, `continuous = true`, `interimResults = true`. Распознанный текст нормализуется (нижний регистр, без пунктуации) и ищется вхождение любого имени из профиля. Имена сравниваются с допуском на одну ошибку, чтобы «Raka» ловилось и как «Kaka». Ограничение: Chrome отправляет звук в облако Google, что нужно отразить в книге как отличие прототипа от целевого устройства.

Классы звуков. YAMNet из `@tensorflow-models` принимает аудио 16 кГц моно, выдаёт вероятности 521 класса каждые ~0.5 с. Карта классов на категории:

Категория	Классы YAMNet
danger	Siren, Civil defense siren, Fire alarm, Smoke detector, Vehicle horn, Car alarm, Emergency vehicle
bell	Bell, Church bell, Chime, Ding, School bell (если есть), Alarm clock
home	Doorbell, Knock, Ding-dong, Door

Порог уверенности берётся из настройки чувствительности: низкая 0.6, средняя 0.4, высокая 0.25. Модель весит около 4 МБ и кэшируется браузером.

8. Хранение (план, шаги 4 и 9)

Локально: zustand с persist в localStorage, ключ `gelang-asistif`. Хранятся профили, активный профиль, настройки, история (последние 200 событий). Firebase добавляется поверх: при входе данные синхронизируются в Firestore в документ пользователя, локальный режим остаётся запасным, чтобы демо работало без сети.

9. Запуск и сборка

```
npm install
npm run dev        # http://localhost:5173
```

```
npm run build      # проверка типов + сборка в dist/
npm run preview    # просмотр собранной версии
```

Деплой (план): `firebase deploy --only hosting`.

10. Проверка на устройстве

Микрофон и вибрация требуют HTTPS или localhost. Для проверки с телефона в одной сети: `npm run dev -- --host` и открыть адрес ноутбука, микрофон при этом Chrome на телефоне не даст; для микрофона нужен задеплоенный HTTPS-адрес или `chrome://flags/#unsafely-treat-insecure-origin-as-secure`.

Скриншоты для документации снимаются headless Chrome через puppeteer-core при ширине 390px.

11. Известные ограничения

- Только Chrome на Android даёт полный набор: речь, YAMNet, вибрация. На iPhone нет Vibration API, Web Speech API ограничен.
- Распознавание в шумном классе неточное; это заложено в книгу как ограничение и компенсируется чувствительностью и ручным вызовом.
- Bluetooth и прошивка ESP32 вне рамок проекта.